

Fundamentos de Arquitectura

Qué es arquitectura · decisiones significativas · ABC · atributos de calidad · cuantificación

Ingeniería de Software II · ITBA · 2026-1C · Repaso conceptual

RESUMEN

La **arquitectura** es el conjunto de **decisiones significativas** (las caras de revertir) que dan estructura al sistema. No vive en abstracto: nace del *Architecture Business Cycle* y se justifica con **atributos de calidad cuantificados** (ISO 25010), traducidos a SLA/SLO/SLI y MTBF/MTTR. Decidir lo irreversible temprano, con el cono de incertidumbre abierto, es la apuesta más cara.

§1 Qué es arquitectura

DEFINICIÓN

Arquitectura: la(s) estructura(s) del sistema — los **elementos de software**, sus **propiedades externamente visibles** y las **relaciones** entre ellos (Bass, Clements, Kazman, *SAIP*). Definiciones convergentes (Kruchten, Booch, Bittner) comparten el mismo núcleo: el conjunto de **decisiones significativas** sobre la organización del sistema (source: arquitectura-software-definicion.md).

La palabra clave: "significativas"

"Significativa" opera como filtro. Una decisión es arquitectónica si:

- **Alto costo de cambio** — revertirla implica rewrites o migraciones de datos.
- **Impacta múltiples atributos de calidad** — no sólo función: toca performance, security, maintainability.
- **Restringe decisiones futuras** — monolito vs microservicios condiciona decenas de decisiones downstream.
- **Es visible al exterior** del módulo — forma parte del contrato con el resto del sistema.

EJEMPLO

Sí es arquitectura: estilo (monolito/microservicios/serverless), persistencia (SQL vs documento, sharding, CQRS), protocolo de integración (REST/gRPC/eventos), modelo de seguridad (OAuth, mTLS, zero-trust), atributos priorizados.

CUÁNDO NO · CUIDADO

No es arquitectura (es diseño): elegir HashMap vs TreeMap dentro de una clase, nombrar una variable, partir un método en dos.

FUENTE

"Architecture is about the important stuff. Whatever that is" (Fowler, "Who Needs an Architect?", IEEE Software 2003). Regla práctica: si un nuevo integrante necesita entenderlo para operar bien, probablemente sea arquitectura.

CUÁNDO NO · CUIDADO

Brooks — No Silver Bullet: no hay decisión arquitectónica universal que resuelva la complejidad esencial. Toda arquitectura es un **compromiso en un contexto**; desconfiar de "los microservicios son mejores" o "clean architecture resuelve X" (source: arquitectura-software-definicion.md).

§2 Architecture Business Cycle (ABC)

DEFINICIÓN

ABC: la arquitectura no es un producto técnico aislado sino el **centro de un ciclo**: las fuerzas del entorno moldean la arquitectura, y la arquitectura resultante **modifica el entorno** que la creó (Bass, Clements, Kazman, *SAIP* cap. 2; source: architecture-business-cycle.md).

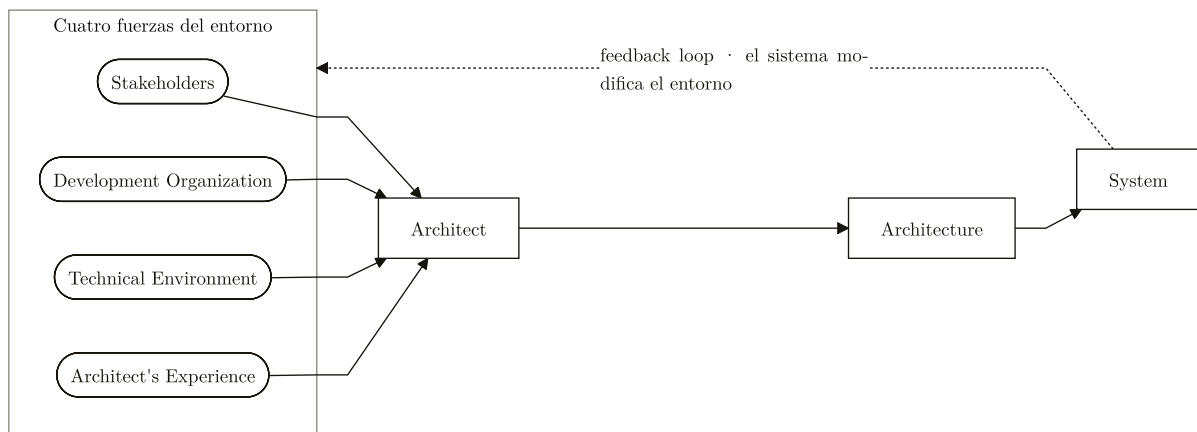


FIGURA 1. *Architecture Business Cycle*: las cuatro fuerzas moldean al arquitecto, que produce la arquitectura y el sistema; el sistema realimenta y modifica el entorno que lo creó.

Inputs — las cuatro fuerzas

Stakeholders

Usuarios, negocio, ops, seguridad, compliance — cada uno presiona por atributos de calidad distintos.

Development Organization

Tamaño del equipo, skills, estructura (Conway's Law), metodología.

Technical Environment

Estado del arte, estándares de industria, legacy a integrar.

Architect's Experience

Sesgos, patrones internalizados, herramientas que sabe usar bien.

FUENTE

*El rol del arquitecto no es "elegir la mejor tecnología": es **sintetizar fuerzas incompatibles en un sistema de decisiones coherente** y explicitar los trade-offs (source: architecture-business-cycle.md).*

CUÁNDO SÍ • VENTAJAS

Por qué importa (implicancias): la arquitectura **óptima no existe en abstracto** — mismos requisitos funcionales producen arquitecturas distintas según org, stack y stakeholders. La arquitectura tiene **esperanza de vida**: lo correcto hoy puede ser obsoleto al moverse el contexto. Por eso se documentan los *drivers*, no sólo la estructura (ver ADR).

CUÁNDO NO • CUIDADO

Cuidado: el diagrama es didáctico — simplifica. En la práctica hay feedback loops más cortos (la primera release ya modifica el entorno), no hace explícito el **factor tiempo** (año 0 ≠ año 5) y asume un arquitecto único cuando suele haber arquitectura emergente de varios.

§3 Atributos de calidad (ISO 25010)

DEFINICIÓN

Atributo de calidad: "a feature or characteristic that affects an item's quality" (IEEE). Son las **propiedades no-funcionales** del sistema — la **moneda con la que se comparan arquitecturas**. Decir "esta arquitectura es mejor" sin nombrar un atributo cuantificable es vacío (source: atributos-de-calidad.md).

AVAILABILITY

PERFORMANCE

SCALABILITY

SECURITY

INTEROPERABILITY

Taxonomía ISO 25000 / 25010

TABLA 1. Run-time qualities — observables cuando el sistema corre

Atributo	Qué mide
Availability	Proporción de tiempo que el sistema está funcional.
Fault Tolerance	Seguir respondiendo (posiblemente degradado) ante fallas de componentes.
Interoperability	Operar con sistemas externos intercambiando información.
Manageability	Que el sysadmin pueda manejar/monitorear la app — salud observable.
Customizability	Que el usuario final personalice look/comportamiento.
Performance	Responsividad — ejecutar acciones dentro de un intervalo temporal.
Precision	Nivel de detalle numérico alcanzable.
Reliability	Mantenerse operativo en el tiempo (sin cortarse).
Scalability	Absorber incrementos de carga sin impacto en performance.
Auditability	Auditar registros y actividades; testear integridad y seguridad.
Security	Prevenir acciones maliciosas/accidentales y disclosure/pérdida de info.

TABLA 2. Design qualities — en la estructura

Atributo	Qué mide
Conceptual Integrity	Coherencia del diseño global.
Maintainability	Facilidad para modificar el sistema.
Portability	Correr en distintos entornos.
Reusability	Componentes reutilizables en otros contextos.

TABLA 3. System / User qualities

Atributo	Qué mide
Supportability	Info útil para diagnosticar fallas (system).
Testability	Facilidad de crear y correr tests (system).
Accessibility	Usable por la mayor cantidad de personas — ally (user).
Usability	Qué tan bien una UI estándar cumple la tarea (user).

Distinciones finas que se confunden en el parcial

EJEMPLO

Availability vs Reliability: un programa que se **reinicia cada 1 h y tarda 1 s** es Available (~99.97%) pero **NO Reliable** (se interrumpe). Operaciones largas e ininterrumpibles (videoconsulta, subasta live, transferencia multi-paso) → domina Reliability; operaciones stateless y cortas (un click HTTP) → suele alcanzar Availability (source: atributos-de-calidad.md).

EJEMPLO

Manageability vs Customizability: Manageability es para el **sysadmin** (salud observable: golden signals — latency, traffic, errors, saturation); Customizability es para el **usuario final** (temas, layouts, idioma). Y **Customizability ≠ Usability**: usability es qué tan bien funciona la UI estándar; customizability es poder *alterarla*.

CUÁNDO NO · CUIDADO

No confundir Interoperability con Fault Tolerance: Interoperability resuelve la **incompatibilidad del protocolo** (tiene dos caras: como cliente y como servidor → adapters/anti-corruption layer por lado); Fault Tolerance responde a la **caída** de un componente. Confundirlos da "soluciones" que ni protegen del fallo ni integran.

EN EL PARCIAL

Si Auditability es prioritario, los logs van en un storage separado con control de acceso distinto (separation of duties: quien opera la app no debe poder borrar/alterar los logs). En el diagrama, el log store es un **componente aparte**, no una tabla en la base operacional (source: atributos-de-calidad.md).

Cuantificación — sin métrica es aspiración, no requisito

Atributo	Métrica típica
Availability	% uptime (99.9%, 99.99%...) → SLA/SLO/SLI.
Reliability	MTBF / MTTR, failure rate.
Performance	Percentiles de latencia (p50, p95, p99), throughput (rps).
Scalability	Usuarios concurrentes, throughput a N nodos.
Security	CVSS, tiempo a parche, blast radius.
Maintainability	Cyclomatic complexity, tiempo de change, % cobertura.
Usability	Task success rate, System Usability Scale (SUS), time-to-task.

TRADE-OFF

Ningún atributo mejora en el vacío: Security ↔ Usability (MFA, timeouts) · Performance ↔ Maintainability (cachés, denormalización) · Availability ↔ Consistency (teorema CAP) · Portability ↔ Performance (capas de abstracción) · Reusability ↔ Simplicity (generalización prematura) · Testability ↔ Deliverability (source: atributos-de-calidad.md).

§4 Cuantificación operacional: SLA / SLO / SLI

DEFINICIÓN

Tríada de Google SRE (*Site Reliability Engineering*, O'Reilly 2016) — estandariza cómo hablar de compromisos operacionales cuantitativos (source: sla-slo-sli.md):

- **SLI** (Indicator) — la **métrica** medible (ej: % de requests 2xx/3xx, latencia p99). Buen SLI = cociente *eventos buenos / totales*, centrado en el usuario.
- **SLO** (Objective) — **objetivo interno**: "el SLI $X > Y$ durante la ventana Z " (ej: 99.9% de requests OK en 30 días rolling).
- **SLA** (Agreement) — **contrato externo** con penalidad (créditos/multas) si se incumple lo publicado.

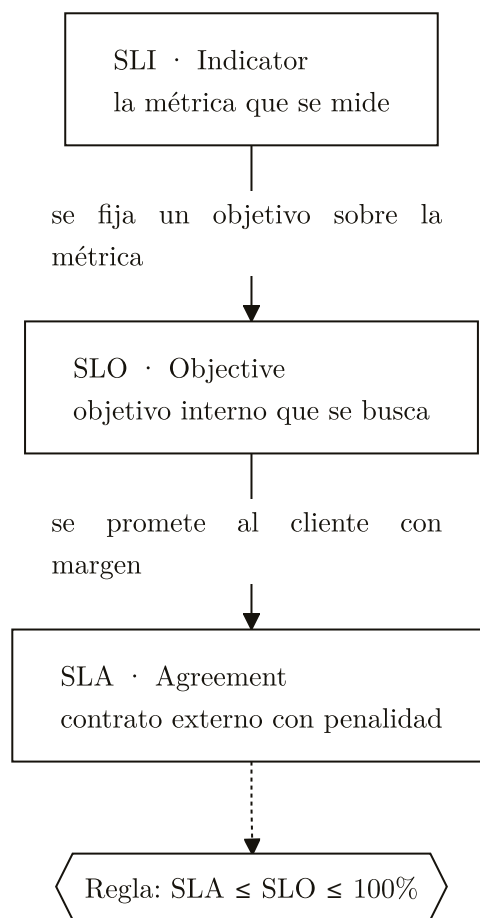


FIGURA 2. De métrica a contrato: el SLI mide, el SLO fija el objetivo interno y el SLA lo promete con penalidad, siempre respetando $SLA \leq SLO \leq 100\%$.

REGLA DE ORO

SLA < SLO siempre. Una empresa opera a 99.99% interno y promete 99.9% al cliente, dejando margen. Publicar SLA = SLO deja sin colchón: el primer incidente genera multa.

EJEMPLO

Error budget: derivado del SLO. Si el SLO es 99.9% mensual, el budget es 0.1% $\approx$ **43 min** de downtime. Es la **moneda de riesgo**: queda budget $\rightarrow$ deploys agresivos; se agotó $\rightarrow$ freeze. Convierte "dev quiere velocidad vs ops quiere estabilidad" en criterio objetivo.

TABLA 4. Los "nines"

Disponibilidad	Downtime mensual
99% (2)	7h 12min
99.9% (3)	43min 49s
99.95%	21min 54s
99.99% (4)	4min 22s
99.999% (5)	26s

Cada "nine" adicional cuesta $\sim 10x$ más en esfuerzo de ingeniería. 5 nines se reserva a infraestructura crítica (telefonía, networking core) (source: sla-slo-sli.md).

CUÁNDO NO • CUIDADO

Errores comunes: SLO sin SLI verificable ("99.9% uptime" sin definir qué es "up"); medir "ping al server" cuando el usuario mide "transacción end-to-end"; SLA = SLO; demasiados SLOs (nadie prioriza); SLOs sin dueño.

§5 MTBF, MTTR y la fórmula de availability

DEFINICIÓN

MTBF — Mean Time Between Failures: tiempo promedio **entre** dos fallas. Mide cuán *confiable* es el sistema. $MTBF = \text{Tiempo total operativo} / \text{Nº de fallas}$.

MTTR — Mean Time To Repair/Recover: tiempo promedio para **restaurar** el servicio tras una falla. Mide cuán *recuperable* es. En SRE moderno "MTTR" = Time To Recover (lo que ve el usuario) (source: mtbf-mttr.md).

REGLA DE ORO

Fórmula clave: $\text{Availability} = MTBF / (MTBF + MTTR)$. Ejemplo: $MTBF = 1000h$, $MTTR = 1h \rightarrow 1000/1001 \approx \underline{99.9\%}$.

CUÁNDO SÍ • VENTAJAS

Dos palancas: subir MTBF (menos fallas: redundancia, testing, validación) o bajar MTTR (recuperar rápido: monitoring, failover automático, runbooks, rollback). La era cloud movió el foco de "que no falle" (MTBF) a "asumir que falla, recuperarse rápido" (MTTR) — design for failure, chaos engineering.

TRADE-OFF

Mismo objetivo, costos distintos: para 99% con $MTTR = 5 \text{ min}$ alcanza $MTBF = 8h$; para el mismo 99% con $MTTR = 1h$ hace falta $MTBF = 100h$ (~ 4 días). **Bajar MTTR es más barato** que subir MTBF — por eso SRE invierte en observabilidad y automation antes que en "zero-bug engineering" (source: mtbf-mttr.md).

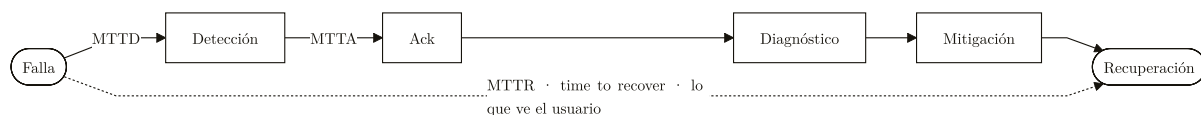


FIGURA 3. Línea de tiempo de un incidente: *MTTD* hasta detectar, *MTTA* hasta reconocer, y *MTTR* como tiempo total de falla a recuperación.

Variantes: **MTTF** (To Failure) para sistemas no reparables; **MTTD** (To Detect) — si es alto, infla el *MTTR*; **MTTA** (To Acknowledge). Cuidado con promediar distribuciones skewed: un incidente de 8h anula 100 de 2 min → usar también p50/p95.

§6 Cono de incertidumbre

DEFINICIÓN

Cono de incertidumbre: el error posible en las estimaciones **no es constante** — se reduce a medida que el proyecto avanza, formando un cono que cierra. Formulado por **Boehm** (COCOMO) y popularizado por **McConnell** (*Software Estimation*, 2006) (source: cono-de-incertidumbre.md).

TABLA 5. Rango de error por momento

Momento	Error típico
Initial product definition	0.25x – 4x
Approved definition	0.5x – 2x
Requirements complete	0.67x – 1.5x
UI design complete	0.8x – 1.25x
Detailed design	0.9x – 1.1x
Implementation	1.0x



FIGURA 4. Cono de incertidumbre: el rango de error en la estimación se va estrechando de 0.25x–4x al inicio hasta 1.0x al implementar, a medida que avanza el proyecto.

Al inicio, "6 meses" puede significar entre 1.5 y 24 meses. Cierra el cono al: comprenderse el dominio, validarse el stack, asentarse el equipo y revelarse las integraciones.

CUÁNDO NO · CUIDADO

El cono NO cierra solo: es una **frontera de posibilidad**, no una trayectoria automática (McConnell). En proyectos mal gestionados la incertidumbre permanece alta o aumenta (scope creep). Cerrarlo exige **acciones deliberadas**: scope explícito, spikes/prototipos, ADRs, feedback temprano. Caso Healthcare.gov: scope creció en silencio pero las decisiones de alto impacto se tomaron con la incertidumbre original (source: cono-de-incertidumbre.md).

REGLA DE ORO

Implicancia arquitectónica: no decidir lo irreversible cuando la incertidumbre es máxima. Estrategias: JEDUF (diseñar sólo lo entendido con evidencia), spikes arquitectónicos, preferir decisiones reversibles, checkpoints con Lightweight ATAM.

EN EL PARCIAL

Respuesta de fundamentos sólida = (1) definir arquitectura por **decisiones significativas**, no por "cajas"; (2) anclar todo en **atributos de calidad cuantificados** (ISO 25010 + métrica); (3) traducir a SLO/SLI y, si hay cliente, SLA; (4) usar la fórmula $Availability = MTBF / (MTBF + MTTR)$ cuando piden disponibilidad; (5) justificar el momento de decidir con el cono. Error que mata: comparar arquitecturas sin nombrar un atributo medible.